Partie 2
Structures classiques

INFO410 – Structures de données classiques

1. Rappel

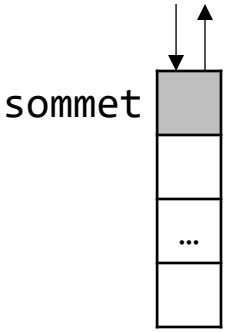
- On s'intéresse aux **collections homogènes** dont la taille peut **varier**
  - Ajouts ou retraits d’éléments
- On sait repérer une **extrémité** de la structure (ou les deux)
- On peut définir le **précédent** et/ou le **suivant** de tout élément qui n'est pas une extrémité
- On doit nécessairement parcourir **séquentiellement** la structure à partir d'une extrémité pour atteindre un élément particulier

2. Les piles

- Ce sont des suites qui correspondent à la notion usuelle de **piles**
  - Exemple : des piles d'assiettes, des piles de livres…
- On retire **toujours** le dernier élément déposé, l'ordre de **sortie** est donc l'inverse de l'ordre d'**entrée**
- Structures dynamiques **homogènes** à un seul point d'accès
- Les données sont ajoutées ou prélevées par l'intermédiaire d'une tête d'accès appelée « **sommet de pile** »

2. Les piles

- Une pile est une structure à accès **LIFO** (Last In, First Out)



- Primitives d'accès
  - `Vide(P)` Teste si la pile P est vide
  - `Empiler(P, X)` Empile un élément X au sommet de la pile P
  - `Depiler(P, X)` Dépile le sommet de la pile P et le met dans X

2. Les piles

- Définition du type « `Pile` »

```
type noeud;
type pile is access noeud;
type noeud is record
   info: T;
   suiv: pile;
end record;
```

2. Les piles

- Vérification si la pile est vide

```
function vide(P: in pile) return boolean is
   …
end vide;
```

2. Les piles

- Ajout d’un élément dans une pile

```
procedure empiler(P: in out pile; X: in T) is
   …
end empiler;
```

2. Les piles

- Suppression d’un élément d’une pile

```
procedure depiler(P: in out pile; X: out T) is
   …
end depiler;
```

3. Les files

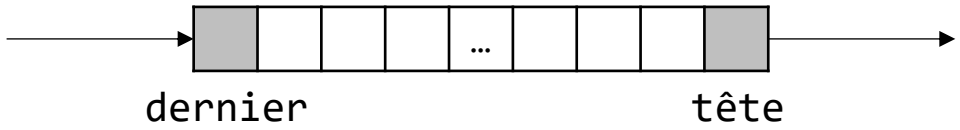
- Ce sont des suites qui modélisent des **files d'attente**
  - Exemple : des clients devant un guichet

- Structures dynamiques **homogènes** avec un accès en **entrée** et un accès en **sortie**

- Les données sont ajoutées par l'intermédiaire d'un poste d'écriture appelé « **dernier** » et prélevées par l'intermédiaire d'un poste de lecture appelé « **tête** »

3. Les files

- Une file est une structure à accès **FIFO** (First In, First Out)



- Primitives d'accès
  - `Vide(F)` Teste si la file F est vide
  - `Ajouter(F, X)` Ajoute un élément X en dernier de la file F
  - `Retirer(F, X)` Retire la tête de la file F et le met dans X

3. Les files

- Définition du type « `File` »

```
type noeud;
type liste is access noeud;
type noeud is record
   info: T;
   suiv: liste;
end record;
type file is record
   tete, dernier: liste;
end record;
```

3. Les files

- Vérification si la file est vide

```
function vide(F: in file) return boolean is
   …
end vide;
```

3. Les files

- Ajout d’un élément dans une file

```
procedure ajouter(F: in out file; X: in T) is
   …
end ajouter;
```

3. Les files

- Suppression d’un élément d’une file

```
procedure retirer(F: in out file; X: out T) is
   …
end retirer;
```

4. Les files version tableau

- La taille est limitée,
  - La file peut s’agrandir jusqu’à dépasser la taille du tableau
  - Même avec moins d’éléments que le tableau, on arrive à un débordement
  - La tête monte tout le temps

- Solution : utiliser un tableau circulaire,
  - Quand le dernier arrive en haut du tableau, on ajoute à partir du bas
  - Problème du test de saturation (tête = dernier)
  - Ajout d’un compteur d’éléments

4. Les files version tableau

- Définition du type « `File` » version tableau

```
type file is record
   tab: array (1..N) of T;
   tete, dernier: 1..N;
   cpt: 0..N;
end record;
```

4. Les files version tableau

- Vérification si la file version tableau est vide

```
function vide(F: in file) return boolean is
   …
end vide;
```

4. Les files version tableau

- Ajout d’un élément dans une file version tableau

```
procedure ajouter(F: in out file; X: in T) is
   …
end ajouter;
```

4. Les files version tableau

- Suppression d’un élément d’une file version tableau

```
procedure retirer(F: in out file; X: in T) is
   …
end retirer;
```